TechMedic v1.5.6 SQLCipher edition

Baseline Technical Specification
Revision 1.0 | October 12, 2026
Development / evaluation release

Purpose and scope

TechMedic uses Jev and OpenAI models (the technology behind ChatGPT) to help diagnose technical issues and recommend solutions for computers, mobile phones, smart devices, televisions, and other electronics. It supports conversational evidence gathering, diagnostic assessment, suggested next steps, and verification of reported results.

The application itself runs on Windows. The types of devices discussed in a case are broader than the host operating system. This is a local, single-user evaluation application, not a production service or an automated device repair agent. Recommendations require human review and execution.

Baseline identity

Attribute	Baseline
Repository	https://github.com/elioli983/TechMedic
Application source baseline	0115463bddfc1eed51483f2afef145e335d0dfa5; this document adds documentation to that application baseline.
License	GPL-3.0-only. Third-party dependencies retain their own licenses.
Tested host	Windows x64; Python 3.14.7
User interface	Streamlit at http://127.0.0.1:8519
Persistence	SQLCipher format 4; Windows current-user DPAPI key protection

Reading guide

2 Architecture and processing; 3 diagnostic semantics and interface; 4 persistence and recovery; 5 security boundaries; 6 installation and configuration; 7 validation, performance, and maintenance. This document records implemented behavior and previously recorded test evidence; it is not a security certification.

Architecture and processing

Component responsibilities

Component	Responsibility
app.py	Streamlit cases, conversation, assessment, live gate, recovery controls, exports, and bounded history.
windows_startup.py	Per-user server-lifetime lock, journal key unlock, live/offline environment gate, and Streamlit invocation.
diagnostic_engine.py; llm.py	Diagnostic orchestration and OpenAI Responses API calls for structured state, research integration, and response composition.
jev.py; status.py	Jev structured evaluations, lifecycle validation, and guarded status/action decisions.
research*.py; safety.py; consistency.py	Gated research with provenance, evidence-preserving integration, safety checks, and consistency checks.
candidate_journal.py; journal_crypto.py	Durable pending input, encryption, processing leases, fencing, retry, and atomic completion.
database.py; database_crypto.py	SQLCipher connections, schema and queries, DPAPI database key lifecycle.
export_bundle.py; presentation.py	Readable diagnostic exports and user-facing labels.

Turn processing contract

1. Capture the submitted input in the encrypted pending journal before external model calls.
2. Claim the submission using an owner identifier, fencing value, and renewable lease.
3. Produce structured diagnostic analysis. Run research only when requested by the analyst and enabled; integrate bounded changes while preserving local evidence.
4. Apply deterministic safety checks, Jev evaluation, lifecycle/readiness/action guards, and response consistency checks.
5. In one transaction, save the completed turn, update case metadata, and mark the journal entry completed.
6. Refresh the conversation and assessment; expose exports only when requested.

Offline mode uses synthetic service implementations. Live mode requires the supported live launcher, the in-app toggle, and provider configuration. Jev scores are decision inputs, not independently calibrated probabilities. Heuristic checks do not guarantee that recommendations are correct or safe.

Diagnostic semantics and interface

Status: diagnostic progress

Displayed status	Meaning
Investigating	Collecting evidence and narrowing the problem.
Possible Diagnosis	A candidate diagnosis has been identified.
Diagnosis Established	Evidence supports an established diagnosis.
Remediation Identified	A corrective action has been identified.
Resolution Verified	Structured post-change evidence supports resolution.

Historical milestones and current remediation readiness are distinct. Escalate is a next action; Escalate (legacy status) is retained only for displaying older records. A proposed fix alone does not establish verified resolution.

Fault domain

Supported choices are Hardware, Operating System, Driver Runtime, Application, Network, Configuration, Peripheral, and unknown. The UI displays unknown as **Not yet determined**; a concrete choice appears under **Likely domain**. An application reporting a symptom is not sufficient evidence that the application is the cause.

TechMedic next action

Ask for information; Collect system information; Run diagnostic test; Verify compatibility; Recommend configuration change; Recommend software repair; Recommend hardware inspection; Verify resolution; Finish diagnosis; Escalate. A dash is shown when no action is available. The displayed effective action can override the initial Jev choice after guardrail checks.

Case operations and history

Users create cases with **+ New case**, continue conversations, select recent cases, and confirm deletion through Recents. Diagnostic Options provides handoff text, a full diagnostic bundle, and developer trace. Case deletion cascades to related turn records through the database foreign key.

Initial history displays the latest 50 turns with one additional row fetched to detect older content. Load older messages expands the window by 50. Switching cases resets the limit. Normal reruns reuse a single bounded history query; requested exports retrieve full history. Completed journal rows are excluded from unfinished-work lists.

Persistence and recovery

Database contract

The UI fixes its database path to **data/techmedic_v1_5_6.sqlcipher** under the application folder. It starts a fresh encrypted database; old SQLite data is ignored, not converted or erased. Standard SQLite is not a fallback. The adapter rejects the plaintext SQLite header and verifies cipher support and schema readability before application queries.

Entity / index	Stored responsibility
cases	Text ID, title, device type, manufacturer, model, operating system, status, created_at, updated_at.
turns	Integer ID, case foreign key, input, response, diagnostic/Jev/safety/research JSON, and creation time.
Pending journal	Submission identity, input fingerprint and encrypted payload, state, attempts, owner/fence, lease expiry, timestamps, and completed-turn linkage.
idx_cases_updated_at	Recent-case ordering by updated_at descending. Created idempotently.
idx_turns_case_id_id	Case-specific turn retrieval ordered by turn ID.

Connection and transaction behavior

Each database operation opens a SQLCipher connection, unlocks the random database key, applies format-4 compatibility, sets temporary SQL storage to memory, checks schema readability, and enables foreign keys. Transactions commit or roll back, and connections close explicitly. The research_json column is added when absent from an existing compatible schema.

Pending submission reliability

Journal states are pending, running, failed, completed, and abandoned. Real-time processing starts lease renewal. Ownership and expiry checks obtain current time after acquiring the write lock. A lost or expired worker cannot commit. Completed turn persistence and journal completion share one transaction. Failed or expired work can be explicitly retried or abandoned; active leases remain protected.

These controls prevent an obsolete worker from committing a turn. They do not provide exactly-once external API calls: retries can incur duplicate provider charges. Input is journaled for recovery, and missing or corrupt original keys cause startup/access failure rather than silent reset.

Encryption and security boundaries

Two protected key lifecycles

Key	Protection and location
Database key	Random 32-byte key, wrapped with current-user Windows DPAPI. Sidecar: data/techmedic_v1_5_6.sqlcipher.key.dpapi.
Pending-input key	Separate Fernet key protected with DPAPI. File: %LOCALAPPDATA%/TechMedic/v1.5.6/journal-key.dpapi.txt. Passed to the server through TECHMEDIC_CANDIDATE_JOURNAL_KEY.

First-run key publication writes protected bytes to a temporary file, flushes them, and renames without overwriting an existing key. Concurrent initializers use the published key. Existing database files or journal sidecars prevent regenerating a missing required key. Database keys are not passed through environment variables, command lines, or logs.

At-rest protection

SQLCipher protects stored case, turn, and pending records; database journal/WAL page contents use encryption. Temporary SQL tables are kept in memory. Automatic unlocking depends on the Windows user profile. The optional cipher_memory_security setting remains at its default after a native Windows failure during earlier validation; database encryption remains enabled.

Transport and local access

Custom Jev endpoints must use HTTPS. Embedded credentials, fragments, invalid ports and malformed endpoints are rejected. Jev requests disable redirects and reject 3xx responses. The supported launcher binds Streamlit to 127.0.0.1 and holds a per-user instance lock for the server lifetime. That lock coordinates startup; it is not authentication.

Boundaries: the application has no browser login. Localhost does not isolate other local users. DPAPI does not protect against software running as the signed-in user. The plaintext .env file, readable exports, old SQLite files, screenshots, in-memory data, pagefiles/crash dumps, and provider-side handling are outside the database-at-rest guarantee.

Backup and failure recovery

Stop TechMedic before backing up the encrypted database, its DPAPI sidecar, and the separate journal-key file. Retain recoverability of the original Windows profile and DPAPI credentials. Copying these files to another account or computer is not a supported portable recovery mechanism. Never delete keys to repair startup. There is no portable recovery password or key-rotation interface in this baseline.

Windows setup and configuration

Supported installation and startup

1. Install Python 3.14 for Windows x64 with the Python launcher.
2. Clone the GitHub repository or extract the source ZIP.
3. Run **INSTALL_WINDOWS.bat**. It creates .venv, installs requirements, verifies SQLCipher, creates data/, and copies .env.example to .env only if absent.
4. Run **START_TECHMEDIC.bat** for the offline synthetic demonstration; no API keys are required.
5. For live evaluation, obtain your own OpenAI and Jev API keys, enter them in .env, stop any offline instance, run **START_LIVE_TECHMEDIC.bat**, and enable the in-app live toggle.

Use http://127.0.0.1:8519. Keep the console open; stop it with Ctrl+C. Restart after configuration changes. Live calls can incur provider charges and send submitted inputs to providers. Use fictional evaluation inputs. A ChatGPT subscription is not a substitute for configuring the OpenAI API key.

Configuration defaults

.env setting	Default / purpose
OPENAI_API_KEY; JEV_API_KEY	User-supplied credentials; empty in the distributed template.
OPENAI_MODEL; JEV_MODEL	gpt-5.1; jev-latest. These are code defaults, not a guarantee of future provider availability.
JEV_API_URL	Blank: route by key type. jv_live_ prefix uses jevtypesafeai.com/api/v1/decide; other keys use api.typesafe.ai/v1/systemone. Both use HTTPS.
TECHMEDIC_WEB_RESEARCH_ENABLED	true; research also requires the analyst to request it.
TECHMEDIC_WEB_RESEARCH_MAX_SOURCES	10; parsed and bounded from 1 to 20.
TECHMEDIC_DB_PATH	Supported by general settings callers; does not override the Streamlit UI database path.

Dependencies and distribution

requirements.txt specifies streamlit >=1.56.0, openai >=2.0.0, requests >=2.32.0, python-dotenv >=1.0.1, cryptography >=45.0.0, and sqlcipher3-wheels ==0.5.7. Most dependencies are minimum versions, so new installs can resolve differently. Internet access and a compatible SQLCipher wheel are required.

The source archive excludes .venv, .env, databases, protected keys, caches, and test outputs. Supported launchers use a server-lifetime lock under LOCALAPPDATA. Stop older instances before updating; direct streamlit run bypasses the guard and is unsupported.

Validation and maintenance baseline

Recorded evidence: October 12, 2026

The recorded Windows/Python 3.14.7 validation passed 190 core/encryption/development-fix tests plus 10 hardening tests (200 total). The tested runtime included SQLCipher 4.12.0 community, SQLite core 3.51.1, Streamlit 1.65.0, and cryptography 50.0.2. These results predate this document; no new full application test run is claimed for this documentation update.

Targeted checks covered heartbeat renewal, lease loss and lock-wait expiry, HTTPS and redirect refusal, bounded history and full exports, index creation, concurrent journal-key initialization, missing/corrupt keys, PowerShell key compatibility, lock contention and forced exit, and live/offline gates. Streamlit AppTest covered startup, paging, complete-history retrieval, and expired-worker retry. An isolated server passed an HTTP health check and competing-startup rejection.

Measured history improvement

Read workload	Median database fetch/decode time
All 1,000 synthetic turns	124.011 ms
Initial 51-row window	8.876 ms

Ten reads per workload; approximately 8 KB of JSON detail per turn on a disposable database. These are workstation-specific fetch/decode measurements, not browser or provider latency guarantees. Query-plan inspection confirmed the recent-case index avoided a temporary sort.

Reproduction commands

Run from the installed project in a normal Windows user session:

```
VALIDATE_OFFLINE_OPTIONAL.bat
.venv\Scripts\python.exe -m unittest -v hardening_tests
.venv\Scripts\python.exe check_sqlcipher_ui.py
.venv\Scripts\python.exe -m pip check
```

Outstanding limits and maintenance

Live provider acceptance was not retested in the four-fix validation. Browser authentication, portable recovery, dependency upgrades, and general production hardening remain outstanding. The wheel is a third-party distribution, not a commercial Zetetic binary. Refer to SQLCIPHER_VALIDATION.md for the recorded dependency advisory review. Do not infer production approval from passing offline tests.

Source and change control

Implementation references: app.py; techmedic/database.py, database_crypto.py, windows_startup.py, candidate_journal.py, config.py, llm.py, jev.py, status.py, and presentation.py. Companion references: README.md, USER_MANUAL.md, SQLCIPHER_STORAGE.md, SQLCIPHER_VALIDATION.md, and docs/ARCHITECTURE.md. FILE_MANIFEST_SHA256.json records release-file hashes. Historical version notes and the v1.0 specification are not current setup instructions.